

Service Desk

Original source: https://docs.gitlab.com/user/project/service_desk/
Date accessed: 2026-06-16

This demo uses publicly available GitLab documentation for educational and portfolio purposes. It is not affiliated with or endorsed by GitLab.

- Tier: Free, Premium, Ultimate
- Offering: GitLab.com, GitLab Self-Managed

Note: This feature is not under active development, but community contributions are welcome. To determine if the feature as it is meets your needs, explore the existing documentation or see the open issues for the Service Desk category to learn more about work that hasn't been done yet. The decision to deprioritize Service Desk has been made to focus on building and extending the work item framework which the Service Desk category will also benefit from long-term. For the information on moving Service Desk into the work item framework, see epic 10772.

With Service Desk, your customers can email you bug reports, feature requests, or general feedback. Service Desk provides a unique email address, so they don't need their own GitLab accounts.

Service Desk emails are created in your GitLab project as new tickets. Your team can respond directly from the project, while customers interact with the thread only through email.

For a video overview, see [Introducing GitLab Service Desk \(GitLab 16.7\)](#).

Service Desk workflow

For example, let's assume you develop a game for iOS or Android. The codebase is hosted in your GitLab instance, built and deployed with GitLab CI/CD.

Here's how Service Desk works for you:

- You provide a project-specific email address to your paying customers, who can email you directly from the application.
- Each email they send creates a ticket in the appropriate project.
- Your team members go to the Service Desk ticket tracker, where they can see new support requests and respond inside associated tickets.
- Your team communicates with the customer to understand the request.
- Your team starts working on implementing code to solve your customer's problem.
- When your team finishes the implementation, the merge request is merged and the ticket is closed automatically.

Meanwhile:

- The customer interacts with your team entirely through email, without needing access to your GitLab instance.
- Your team saves time by not having to leave GitLab (or set up integrations) to follow up with your customer.

Related topics

- [Configure Service Desk](#)
- [Improve your project's security](#)
- [Customize emails sent to external participants](#)

- Use a custom template for Service Desk tickets
- Support Bot user
- Default ticket visibility
- Reopen tickets when an external participant comments
- Custom email address
- Use an additional Service Desk alias email
- Configure email ingestion in multi-node environments
- Use Service Desk
 - As an end user (ticket creator)
 - As a responder to the ticket
- Email contents and formatting
- Convert a regular issue to a Service Desk ticket
- Privacy considerations
- External Participants
- Service Desk tickets
 - As an external participant
 - As a GitLab user

Troubleshooting Service Desk

Emails to Service Desk do not create tickets

- Your emails might be ignored because they contain one of the email headers that GitLab ignores.

- Emails might get dropped if the sender email domain is using strict DKIM rules and there is a verification failure due to forwarding emails to the project-specific Service Desk address. A typical DKIM failure message, which can be found in email headers, might look like:

```
dkim=fail (signature did not verify) ... arc=fail
```

The exact wording of the failure message may vary depending on the specific email system or tools in use. Also see this [article on DKIM failures](#) for more information and potential solutions.

Email ingestion doesn't work in 16.6.0

GitLab Self-Managed 16.6.0 introduced a regression that prevents mail_room (email ingestion) from starting. Service Desk and other reply-by-email features don't work. Issue 432257 tracks fixing this problem.

The workaround is to run the following commands in your GitLab installation to patch the affected files:

```
curl --output /tmp/mailroom.patch --url "https://gitlab.com/gitlab-org/gitlab/-/merge_requests/137279.diff"
patch -p1 -d /opt/gitlab/embedded/service/gitlab-rails < /tmp/mailroom.patch
gitlab-ctl restart mailroom

curl --output /tmp/mailroom.patch --url "https://gitlab.com/gitlab-org/gitlab/-/merge_requests/137279.diff"
cd /opt/gitlab/embedded/service/gitlab-rails
patch -p1 < /tmp/mailroom.patch
gitlab-ctl restart mailroom
```

Attribution

Source title: Service Desk

Original source: https://docs.gitlab.com/user/project/service_desk/

Raw public source used for conversion:

https://gitlab.com/gitlab-org/gitlab/-/raw/master/doc/user/project/service_desk/_index.md

Date accessed: 2026-06-16

Generated for a DocuQuery AI public-document RAG case study. No private, leaked, scraped personal, or customer-confidential data was intentionally included.